WNDRVR
SUPPORT NETWORK

VXWORKS 7 BUILDING CUSTOM LAYERS WITH DEPENDENCIES TUTORIAL, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 Building Custom Layers With Dependencies Tutorial, SR0650

25 August 2020

1. BUILDING CUSTOM LAYERS WITH DEPENDENCIES

VxWorks® layers often depend on software APIs being present in the VSB to build. This tutorial shows the workflow for creating layers with dependencies on the software API exposed by other layers.

Wind River layers rarely exist in isolation. Often, the code contained within a layer depends on code from other layers being available at build time. For example, your layer may require the Wind River USB stack to be present to function. Or you may develop a couple of co-dependent custom layers. You ensure build time availability of layers by declaring a dependency between layers.

This tutorial explains the workflow for layer dependency and is as follows:

1. Create a custom layer.
2. Create a custom layer with dependencies on both custom and Wind River supplied layers.
3. Test the VIP built with the VSB containing dependent layers.

2. EXAMPLE: CREATING AN INSIDE CUSTOM LAYER AND INTEGRATING IT INTO A VSB

You can create a custom layer within the VxWorks source tree and integrate it into your VxWorks source build (VSB) project using the project build system.

Before You Begin

This example assumes you have access to the **cobble** downloadable kernel module (DKM) example project at *installDir* **\vxworks-7\samples\dkm**.

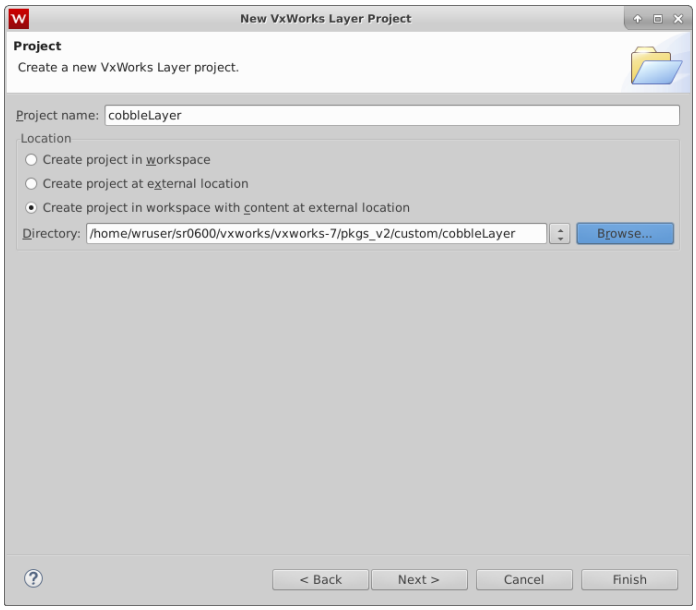
It also assumes you have a pre-existing VxWorks source build (VSB) and VxWorks image project (VIP) present in Workbench. Follow the steps below using a VxWorks simulator (VxSim) instance as the target.

About This Task

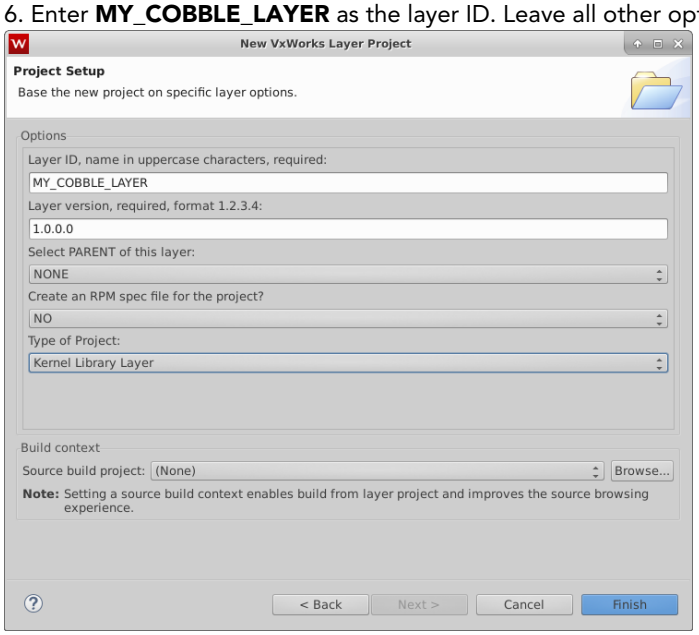
For this tutorial, you use the Workbench **cobble** example DKM project to represent the application code you want to integrate into a layer. Create a layer project, insert the **cobble** DKM application code into it, rebuild the VSB and VIP, and check that the target boots and runs.

Procedure

1. Create a folder for your new layer projects.
    1. Create a root directory within the Wind River installation at *installDir* **\vxworks-7\pkgs_v2\custom**.
    2. Create a folder for your new custom cobble layer project at *installDir* **\vxworks-7\pkgs_v2\custom\cobbleLayer**.
2. Create a VxWorks layer project in your new layer project folder and call it **cobbleLayer**.
    1. From the Workbench menu, select **File > New > Wind River Workbench Project**.
    2. Select the option VxWorks 7 as the target operating system from the drop-down menu if you are prompted to choose the **Target Operating System**.
    3. From the Build type drop-down list, select **Layer**. Click **Next**.
    4. Give the Project the name **cobbleLayer**.
    5. Under the **Location** dialog, select **Create project in workspace with content at external location** and browse to the layer folder. Click **Next**.



6. Enter **MY_COBBLE_LAYER** as the layer ID. Leave all other options as default and click **Finish**.



3. Populate **cobbleLayer** with **cobble.c**.
    1. Go to *installDir* **\vxworks-7\samples\dkm\cobble**.
    2. Right-click on the **cobble.c** source file and select **Copy**.
    3. In the project explorer, navigate to **cobbleLayer**. Highlight the **src** folder within the **cobbleLayer** project and click **Paste**.
4. Update the **cobbleLayer** metadata file.
    1. Locate the file **layer.vsbl** at the root of the **cobbleLayer** project. Open the file for editing.
    2. Locate the line beginning with the tag **SYNOPSIS**. Replace the remaining text on this line with an appropriate description. For example, "This layer implements Cobble functionality".

```
Layer MY_COBBLE_LAYER {
SYNOPSIS This layer implements Cobble functionality
VERSION 1.0.0.0
```

    3. Save the file.

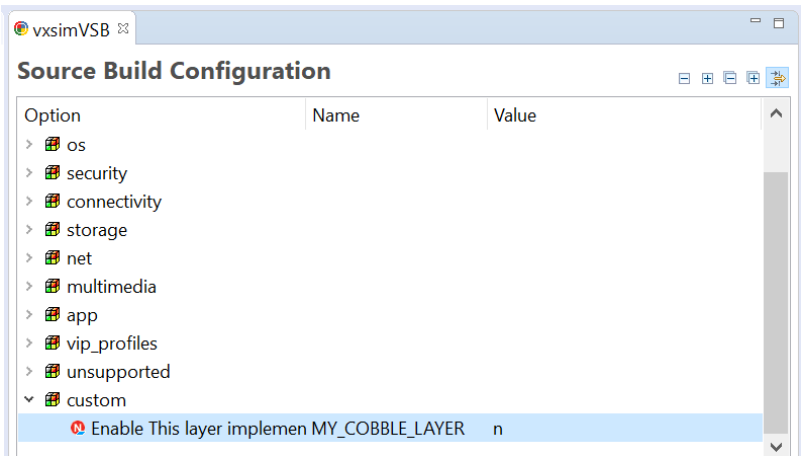
5. Use a host terminal to check that **cobbleLayer**, otherwise known as **MY_COBBLE_LAYER**, does exist.

```
C:\WindRiver\> wrenv -p vxworks-7
C:\WindRiver\> wrtool -data C:\WindRiver\workspace
Active workspace is 'C:\WindRiver\workspace'.
WrTool, version 4.1.0
Copyright (c) 2013 - 2014 Wind River Systems, Inc.


C:\WindRiver\> prj vsb info MY_COBBLE_LAYER
Name: MY_COBBLE_LAYER
[...]
[...]
Location: C:/WindRiver/vxworks-7/pkgs_v2/custom/cobbleLayer
```

6. Verify that your VSB has access to **MY_COBBLE_LAYER** by opening the VSB source build configuration in the project explorer.

   You can see **MY_COBBLE_LAYER** appearing as a component within the Source Build Configuration and the **custom** container.



7. Include **cobbleLayer** in your VSB.
   1. Right-click on the red component within **custom** and select **Add with Dependencies** to add **MY_COBBLE_LAYER** to the VSB.
   2. Observe **MY_COBBLE_LAYER** is now a green component.
   3. In the project explorer, right-click on the VSB and select **Rebuild Project**.
8. Modify your VIP to start the cobble application upon target startup.
   1. Locate the VIP related to your VSB in the project explorer.
   2. Locate the **usrAppInit.c** file within the project and open it for editing.
   3. Insert a call to the **cobble.c** progStart( ) function into the **usrAppInit.c** usrAppInit( ) function.

```
/*******************************************************************************
*
* usrAppInit - initialize the users application
*/

void usrAppInit (void)
    {
#ifdef  USER_APPL_INIT
    USER_APPL_INIT;     /* for backwards compatibility */
#endif

    /* TODO: add application specific code here */
    progStart();
    }
```

9. Rebuild the VIP and reboot the target from the rebuilt VIP. Check that the expected cobble tasks are running on the target from the VxWorks shell.

   The **cobble.c** file is a simple multi-tasking application that simulates a data collection system in which data is retrieved from an external source (for example, a device that generates interrupts as data comes in). The four cobble tasks are:

   - tCosmos, which simulates an interrupt service routine (ISR) and generates new data.
   - tSchlep, which collects the data.
   - tCrunch, which performs calculations on the data to obtain a result value.
   - tMonitor, which monitors the results and prints a warning message if they are outside of the safety range.

   > **Note:** If you do not add progStart( ) to **usrAppInit.c**, cobble does not start upon target reboot and **cobbleLayer** does not link into the kernel image. Rather, the layer is omitted by the VxWorks build system because there is no reference to the **cobble.c** application in the VxWorks BSP or application code. As a consequence, the VxWorks build system linker omits **cobbleLayer** during the link phase of the VIP build.

```
-> i

  NAME         ENTRY       TID    PRI   STATUS      PC       SP     ERRNO  DELAY
----------  ------------ -------- --- ---------- -------- -------- ------- -----
tIsr0       102678f0     3800da08   0 PEND       102714ee 380cff60       0     0
tJobTask    101a82b0     38049e08   0 PEND       102714ee 3813ff00       0     0
tExcTask    101a7ca0     103dfb40   0 PEND       102714ee 103e5d40       0     0
tLogTask    logTask      3804dd48   0 PEND       1026f073 3817feb0       0     0
tShell0     shellTask    38369d40   1 READY      1027a760 3841fc40       0     0
tErfTask    100ed8c0     38002e10  10 PEND       10271e1f 3802ff00       0     0
ipcom_tick> 1003a1b0     38437e48  20 PEND       102714ee 384dfed0       0     0
tVxdbgTask  100ffc70     3832b4b0  25 PEND       102714ee 3838ff20       0     0
tTcfEvents  1020f1f0     103f8380  49 PEND+T     102714ee 103f81f0       0    54
tTcf        1015a0b0     383ffb18  49 PEND       102714ee 3845f890       0     0
tTcf        1015a0b0     38433a10  49 PEND       102714ee 3849f8a0       0     0
tTcf        1015a0b0     38479908  49 PEND       102714ee 3861f8f0       0     0
tTcf        1015a0b0     3847d5c8  49 PEND       102714ee 3865fa40       0     0
tNet0       ipcomNetTask 38008b30  50 PEND       102714ee 3806fef0       0     0
tAioIoTask> aioIoTask    3811b440  50 PEND       10271e1f 381ffef0       0     0
tAioIoTask> aioIoTask    3811e870  50 PEND       10271e1f 3823fef0       0     0
ipcom_sysl> 10038c10     38086280  50 PEND       10271e1f 382bfe20       0     0
tNetConf    10075840     380a5ec8  50 PEND       102714ee 3834fcf0       0     0
tAnalysisA> cafe_event_> 103d9ca0  50 PEND       102714ee 103d9b30       0     0
tAioWait    aioWaitTask  38117f10  51 PEND       102714ee 381bfe80       0     0
tCosmos     cosmos       38438518 200 DELAY      10279812 3851ff80       0     1
tSchlep     schlep       38438948 220 PEND       102714ee 3855ff20       0     0
tMonitor    monitor      3843f900 230 READY      10149800 385dffb0       0     0
tCrunch     crunch       3843f5d0 240 READY      100b5ae0 3859ffd0       0     0
ipcom_egd   10037c10     38089998 255 READY      10037c80 382ffec0       0     0
value = 0 = 0x0
->
```

3. EXAMPLE: BUILDING CUSTOM LAYERS WITH DEPENDENCIES

Custom VxWorks layers can be configured with dependencies. When a complex layer requires particular API functions, dependencies ensure these functions are present at VSB build time.

Before You Begin

This example assumes you have already completed Example: Creating an Inside Custom Layer and Integrating it into a VSB on page 2. That example results in a VxWorks layer project called **cobbleLayer** that is located internal to the Wind River installation directory and also present in the Workbench project explorer.

The project explorer should also already contain a **hello_VxWorks** downloadable kernel module (DKM) example project.

Your must also have a pre-existing VxWorks source build (VSB) and VxWorks image project (VIP) available. Use a VxWorks simulator (VxSim) instance as the target.

About This Task

In this example you will build a configuration of multiple, inter-dependent layers.

Procedure

1. Prepare your VxWorks installation for your layer projects.
    1. Create a root folder for your layers at *installDir* **\vxworks-7\pkgs_v2\custom**.
    2. Create a folder for your **helloLayer**project at *installDir* **\vxworks-7\pkgs_v2\custom\helloLayer**.
2. Create a VxWorks layer project in your new layer project folder and call it **helloLayer**.
    1. From the Workbench menu, select **File > New > Wind River Workbench Project**.
    2. From the **Build Type** drop-down list, select **Layer**. Click **Next**.
    3. Name the project **helloLayer**.
    4. Under the **Location** dialog, select **Create project in workspace with content at external location** and browse to the layer folder: *installDir* **\vxworks-7\pkgs_v2\custom\helloLayer**. Click **Next**.
    5. Enter **MY_HELLO_LAYER** as the layer ID. Leave all other options as default. Click **Finish**.
3. Populate **helloLayer** with **hello.c**.
    1. In the Project Explorer view, expand the **hello** project.
    2. Right-click on the **hello.c** source file and select **Copy**.
    3. In the project explorer, navigate to **helloLayer**. Highlight the src folder within the **helloLayer** project and select **Paste**.
4. Prepare the **helloLayer** API.
    1. In the Workbench editor, open the **hello.c** file that you just copied.
    2. Change the name of the function from main( ) to hello_world( ). This makes referencing this function from the cobble layer easier.

```
1  /* hello.c - hello world example */
2
3  /* Copyright 2004 Wind River Systems, Inc. */
4
6  modification history
10
11 #include <stdio.h>
12
13 int hello_world(void)
14 {
15     printf("Hello World!\n");
16     return 0;
17 }
```

5. Declare a layer dependency in the **helloLayer** metadata file.
   1. Locate the file **layer.vsbl** at the root of the **helloLayer** project. Open the file for editing.
   2. Locate the line beginning with the tag **SYNOPSIS**. Replace the remaining text on this line with an appropriate description. For example, "This layer implements Hello functionality".
   3. Locate the line that starts with the tag **LAYER_REQUIRES**. Add **SYSTEMVIEWER_AGENT** to this line.

```
Layer HELLO_LAYER {
SYNOPSIS This layer implements Hello functionality
VERSION 1.0.0.0
VSB_REQUIRES
LAYER_REQUIRES   SYSTEMVIEWER_AGENT
```

   4. Once the file is updated, save **layer.vsbl**.

> Note: In this step, you are adding a dependency to Wind River's system viewer agent to show that custom layers can be dependant on Wind River layers as well as other custom layers.

6. Declare a layer dependency in the **cobbleLayer** metadata file.
   1. Locate the file **layer.vsbl** at the root of **cobbleLayer** project. Open the file for editing.
   2. Locate the line that starts with the tag **LAYER_REQUIRES**. Add **HELLO_LAYER** to this line.

```
Layer MY_COBBLE_LAYER {
SYNOPSIS This layer implements Cobble functionality
VERSION 1.0.0.0
VSB_REQUIRES
LAYER_REQUIRES   HELLO_LAYER
```

> Note: For layers with complex dependency trees, you may need to set **FASTBUILD NO** to disable parallel layer build support for this layer, and avoid Workbench build problems.

   3. Once this file is updated, save **layer.vsbl** and close the editor.
7. Modify the **cobbleLayer** file **cobble.c** so that it references **helloLayer**.
   1. From the Workbench project explorer, open the **cobble.c** file in the editor.
   2. Locate the entry point to the body of the cobble progStart( ) function.
   3. Add a call to hello_world( ) just before the line of code that spawns the **tCosmos** task.

```
STATUS progStart (void)
    {
    syncSemId = semBCreate (SEM_Q_FIFO, SEM_EMPTY);
    dataSemId = semBCreate (SEM_Q_FIFO, SEM_EMPTY);

    nodeListGuardSemId = semMCreate (  SEM_Q_PRIORITY
                | SEM_INVERSION_SAFE
                | SEM_DELETE_SAFE);

    pCurrNode = NULL;   /* just in case */

    hello_world();
    /* get started */

    tidCosmos = taskSpawn ("tCosmos", 200, 0, STACK_SIZE,
        (FUNCPTR) cosmos,0,0,0,0,0,0,0,0,0,0);

    tidSchlep = taskSpawn ("tSchlep", 220, 0, STACK_SIZE,
        (FUNCPTR) schlep,0,0,0,0,0,0,0,0,0,0);

    /*
     * priority mis-assignment provides desired educational
     * malfunction of not allowing tCrunch to run because
     * tMonitor is higher priority
     */

    tidCrunch = taskSpawn ("tCrunch", 240, 0, STACK_SIZE,
        (FUNCPTR) crunch,0,0,0,0,0,0,0,0,0,0);

    tidMonitor = taskSpawn ("tMonitor", 230, 0, STACK_SIZE,
        (FUNCPTR) monitor,0,0,0,0,0,0,0,0,0,0);

    return (OK);
    }
```

8. Add **helloLayer** and **cobbleLayer** to your VSB.
    1. In the project explorer, open the VSB source build configuration.
    2. Verify that you can see **cobbleLayer** appearing there as a root component in the Source Build Configuration.
    3. Right-click on the red component within **cobbleLayer** and select **Add with Dependencies** to add it to the VSB.

    Notice that Workbench responds with a dialog box indicating that both **cobbleLayer** and **helloLayer** are being added to the VSB.

    > Note: You may notice that the **SYSTEMVIEWER_AGENT** is not mentioned in the dialog box as a layer to be added to the VSB. This is because this layer is already included in the VSB.

    4. In the project explorer, right-click on the VSB and select **Rebuild Project**.
9. Modify your VIP so that **cobbleLayer** starts on target reboot.
    1. Locate the VIP related to your VSB in the project explorer.
    2. Find the **usrAppInit.c** file within the VIP project and open it for editing in Workbench.
    3. If it does not already exist, place a call to the **cobble.c** function progStart( ) in the function usrAppInit( ).

```
/*******************************************************************************
*
* usrAppInit - initialize the users application
*/

void usrAppInit (void)
    {
#ifdef  USER_APPL_INIT
    USER_APPL_INIT;     /* for backwards compatibility */
#endif

    /* TODO: add application specific code here */
    progStart();
    }
```

10. Rebuild the VIP and reboot the target.
11. On the target console, notice that the **cobbleLayer** tasks are running, and **cobbleLayer** has called the dependant layer, **helloLayer**.

    The **cobble.c** file is a simple multi-tasking application that simulates a data collection system in which data is retrieved from an external source (for example, a device that generates interrupts as data comes in). You have modified the root cobble function to call hello_world( ). The four cobble tasks are:

- tCosmos, which simulates an interrupt service routine (ISR) and generates new data.
- tSchlep, which collects the data.
- tCrunch, which performs calculations on the data to obtain a result value.
- tMonitor, which monitors the results and prints a warning message if they are outside of the safety range.

Note: If you do not add progStart( ) to **usrAppInit.c**, cobble does not start up on target reboot. Furthermore, **cobbleLayer**does not even link into the kernel image. It is ommitted by the VxWorks build system because there is no reference to the **cobble.c** application in the VxWorks BSP or application code.

additionally, if you do not add the function call to hello_world( ) from within progStart( ), the **hello_world** library function is not linked during the Workbench VSB build even though a dependency is specified between **cobbleLayer** and **helloLayer**.

```
        \77777777\   -      Build date: Jun  4 2015 14:07:29
         \77777777\
          \7777777/         Copyright Wind River Systems, Inc.
           \77777/   -                  1984-2015
            \777/   /7\
             \7/   /777\
              -   -------

             Board: SIMNT board
     OS Memory Size: 511MB
   ED&R Policy Mode: Deployed
       Debug Agent: Started (always)

Loading symbol table from host:D:/WindRiver/workspace/VIP/default/vxWorks.sym ...done

Hello World!
-> i

  NAME         ENTRY       TID    PRI   STATUS      PC       SP     ERRNO  DELAY
----------  ------------ -------- --- ---------- -------- -------- ------- -----
tIsr0       102558b0     3800d810   0 PEND       1025ebde 380cff60       0     0
tJobTask    1019b890     38049ab8   0 PEND       1025ebde 3813ff00       0     0
tExcTask    1019b280     103ab460   0 PEND       1025ebde 103b1660       0     0
tLogTask    logTask      3804d9f8   0 PEND       1025c763 3817feb0       0     0
tShell0     shellTask    3836b6b0   1 READY      10267e50 3844fc40       0     0
tErfTask    100e1d80     38002e10  10 PEND       1025f50f 3802ff00       0     0
ipcom_tick> 1002e970     38083660  20 PEND       1025ebde 3850fed0       0     0
tVxdbgTask  100f4150     3832ad80  25 PEND       1025ebde 3838ff20       0     0
tTcfEvents  101fd1b0     103c19a0  49 PEND+T     1025ebde 103c1810  3d0004   433
tTcf        1014d690     3836f2b0  49 PEND       1025ebde 3848f8a0       0     0
tTcf        1014d690     3842bd70  49 PEND       1025ebde 384cfa40       0     0
tTcf        1014d690     38461b38  49 PEND       1025ebde 3864f8f0       0     0
tTcf        1014d690     384658f8  49 PEND       1025ebde 3868f890       0     0
tNet0       ipcomNetTask 38008b30  50 PEND       1025ebde 3806fef0       0     0
tAioIoTask> aioIoTask    3811b1d0  50 PEND       1025f50f 381ffef0       0     0
tAioIoTask> aioIoTask    3811e510  50 PEND       1025f50f 3823fef0       0     0
ipcom_sysl> 1002d3d0     38086030  50 PEND       1025f50f 382bfe20       0     0
tNetConf    1006a000     380a5b70  50 PEND       1025ebde 3834fcf0       0     0
tAnalysisA> cafe_event_> 103a55c0  50 PEND       1025ebde 103a5450       0     0
tAioWait    aioWaitTask  38117e90  51 PEND       1025ebde 381bfe80       0     0
tCosmos     cosmos       38083d58 200 DELAY      10266f02 3854ff80       0     2
tSchlep     schlep       38084188 220 PEND       1025ebde 3858ff20       0     0
tMonitor    monitor      38084a18 230 READY      100b7b60 3860ffb0       0     0
tCrunch     crunch       380845d0 240 READY      100aa280 385cffd0       0     0
ipcom_egd   1002c3d0     38089648 255 READY      1002c440 382ffec0       0     0
value = 0 = 0x0
-> 
```